One successor step for equality and computation

A five-rule clausal presentation of intuitionistic primitive recursive arithmetic

Prepared for Vladimir Reshetnikov

Abstract

We give a simpler arithmetic inference interface for the current `ConciseIPRA` proposal. On its already available zero-normal iterator syntax, reflexivity and one uniform successor-evaluation rule derive both computation and arbitrary equational transport. The three syntactic alternatives of the current computational-equality rule are replaced by one. The key is an explicit two-step derivation of equality composition using a one-step identity iterator.

Together with the existing clausal logical interface, the complete presentation has five primitive rule schemes: Use, Refl, Step, DE, and DI. Its formulas are only equations and one two-variable binder. We prove exact consequence equivalence with the native kernel and therefore whole-domain bi-interpretability and equiconsistency with full first-order iPRA. Both numerical composites have the identity graph $x = y$.

The fully validated standalone Python checker measures 2950 normalized bytes, 829 significant tokens, and 902 AST nodes. A matched zero-normal computation/transport control measures 3048, 860, and 949, respectively. The comparison counts normalization, syntax validation, scope handling, clause storage, and the command-line interface. The contribution is a local mathematical and implementation reduction; no absolute minimum is asserted. The new equivalence is a written proof supported by executable proof translations and boundary tests, not a new Lean or Rocq verification.

Frozen comparison. `VladimirReshetnikov/ProveIt`, commit `6ca328039cafa1537d81ee6b67ee2dc54ffa5a4e`.

Scope. Full intuitionistic first-order arithmetic, including quantified formulas and arbitrary retained assumptions.

Contents

1 Result and comparison with the current proposal **3**
1.1 Precise claims . . . 3
1.2 What the simplicity claim establishes . . . 4
1.3 Attribution and proof status . . . 4

2 Uniform successor evaluation on zero-normal terms **4**
2.1 Canonical syntax and what it costs . . . 4
2.2 One evaluation pattern suffices for equality and computation . . . 5
2.3 A normal zero witness for the single introduction . . . 6

3 A five-pattern clausal calculus **6**
3.1 Formulas and retained clauses . . . 6
3.2 Complete rule inventory . . . 7
3.3 Structural bookkeeping . . . 7
3.4 Conditional closures as formulas . . . 8
3.5 Recovering every case introduction . . . 8

4 Exact equivalence and the inherited iPRA bridge **9**

5 Complete verifier and certificate contract **10**
5.1 Concrete syntax . . . 10
5.2 Why the opening operation needs no shift . . . 11
5.3 Measurements . . . 11
5.4 Verification evidence and its limits . . . 12

6 Full first-order interpretation and research boundary **12**
6.1 What remains open . . . 13

A Inherited proof material and attribution **14**

B Complementary introductions with one local assumption **14**
B.1 Complete native syntax and rule inventory . . . 14
B.2 Exact equivalence and the unused-witness construction . . . 15
B.3 Induction and numerical cases separate cleanly . . . 16
B.4 The entire ordinary logical interface . . . 17
B.5 Whole-domain realizations and both formula composites . . . 18
B.6 What the theorem does and does not claim . . . 19

C Arithmetic strength and both ordinary interpretation composites **20**
C.1 The arithmetic dependency order 20

D Internal scalar predecessor bootstrap **22**
D.1 Elementary terms and two finite cycles 22
D.2 A ramp with a global scaling identity 23
D.3 A comparator from scalar iteration 23
D.4 Saturated iteration gives predecessor 24
D.5 Dependency boundary 25

E From literal-equation induction to full intuitionistic PRA **26**
E.1 The exact hypotheses 26
E.2 Constructor separation and two elementary tests 27
E.3 Four elementary uses of equation induction 28
E.4 An equation invariant that reflects actual equality 28
E.5 Compilation of all quantifier-free formulas 30
E.6 Recovery of induction and the full theory 31

F Pairing, primitive recursion, and internal numerical composites **32**
F.1 Pairing from a bounded square-root search 32
F.2 Compiling each primitive recursion 33
F.3 Why both numerical round trips are internal 33

G Complete primary verifier **34**

1 Result and comparison with the current proposal

The useful simplification is the following single arithmetic inference. Write $\overline{I}$ for the canonical iterator constructor, which returns its initializer at the literal counter zero and otherwise retains an iterator node:

$$\frac{\overline{I}(c, a; u.h) = t \qquad I(Sc, a; u.h) = r}{r = N(h[t/u])}\ \text{Step}.$$

Here N contracts zero-counter iterators recursively, and substitution is capture-free. The rule reads its counter, initializer, and body from the second premise. Its first premise is required to have exactly the corresponding predecessor source. No arbitrary equality template or occurrence path is supplied in the certificate.

In the frozen repository, `NativeEval.lean` implements the six-rule computational-equality presentation with separate `evalId`, `evalZero`, and `evalSucc` constructors for its three Eval alternatives. The repository also contains zero-normalization and initializer-only equality proposals. The incoming `ipra_clausal_canonical.zip` already has a five-rule clausal calculus, and `ipra_scalar_uniqueness.zip` already replaces induction by uniqueness of iteration. Both archives were inspected before selecting this construction. The present proposal does not claim those earlier ideas as new.

The new observation is that on zero-normal initialized iterators, *the successor alternative alone recovers the missing identity alternative.* For $J_s = I(S0, s; v.v)$, two Steps give

$$\frac{s = t \qquad \dfrac{\dfrac{s = r \qquad J_s = J_s}{J_s = r}\ \text{Step}}{r = t}\ \text{Step}.}{} \tag{1}$$

This equality-composition rule works under arbitrary retained assumptions. It does not depend on induction, numerical cases, or the logical binder. Together with one-step application of a numerical template, it restores the entire equality interface.

1.1 Precise claims

Theorem 1.1 (Main result)**.** *Let C_{step} be the five-rule calculus specified in Section 3. It is a faithful presentation of a theory bi-interpretable with full first-order iPRA. The interpretation domains are the whole numerical domains, interpreted equality is actual equality, and both composite interpretations have isomorphism graph $x = y$. It is consequently equiconsistent with iPRA.*

The new arithmetic inference interface is interderivable, under arbitrary side assumptions, with normalized successor computation and one-template equational transport. Replacing the latter pair by Refl and Step therefore preserves the consequence relation, not merely the set of true standard numerical equations.

The proof has three layers. First, the equality argument (1) recovers computation and transport. Second, the clause interface recovers the native conditional-closure binder and both case and progress introductions. A nonliteral canonical term provably equal to zero handles the one delicate introduction case. Third, the existing internal arithmetic bridge transfers the result to iPRA. The first two layers are proved in full below; the exact inherited bridge proofs are reproduced in the appendices with attribution.

1.2 What the simplicity claim establishes

There are several distinct comparisons, and they should not be collapsed into one number:

- Against the current three-alternative Eval rule, Step has one arithmetic pattern. The zero equation is accounted for by the existing canonical syntax.
- Against a matched zero-normal clausal calculus, the number of inference schemes remains five. Its explicit computation axiom and template-taking transport rule are replaced by Refl and Step. The certificate no longer carries either a computation triple or a transport template for those steps.
- Against the incoming five-rule clausal canonical checker, the new complete checker is smaller in all three reported source coordinates. This comparison also changes the numerical representation, so the matched control is the cleaner measurement of the arithmetic change.

This report does not beat every historical source-size profile. The repository contains an aggressively encoded 516-byte guarded profile, and the incoming uniqueness package has an 866-byte fully validated implementation of a different certificate protocol. The new readable checker is not smaller than those programs. There is no representation-independent theorem here about the globally simplest axiomatization. The established result is the exact rule simplification and its controlled complete-checker reduction.

1.3 Attribution and proof status

The scalar iterator, the D binder, clause contexts, the witness-directed introduction, and zero-normalization are inherited from the supplied repository. The contribution of this package is the uniform Step reduction, its combination with the existing logical interface, the written equivalence proof, and its executable implementation and translations.

The baseline's Lean modules establish the inherited native/scalar/iPRA bridge. We inspected those sources but did not compile a new Lean formalization in this package. Passing Python certificates supplies implementation evidence; the universal metatheorem is the written proof, and the two kinds of evidence are kept separate.

2 Uniform successor evaluation on zero-normal terms

2.1 Canonical syntax and what it costs

Start with raw scalar terms

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h),$$

where u binds only its occurrences in h. Define N by structural recursion, normalizing all immediate subterms and replacing $I(0, a; u.h)$ by a whenever the normalized counter is literally zero. Only this zero rule is contracted. In particular, $I(S0, a; u.h)$ is retained and is not definitionally $h[a/u]$.

The numerical terms of the new calculus are the fixed points of N. Every displayed substitution means ordinary capture-free substitution followed by N; write $h\langle t\rangle = N(h[t/u])$. Normalization applies also under all term and formula binders. On formulas it changes numerical terms only. Freshness is computed in the usual free-variable sense.

The elementary coherence equations are

$$\begin{aligned} N(N(t)) &= N(t), \\ N(t[\sigma]) &= N(N(t)[N \circ \sigma]), \\ (t\langle\sigma\rangle)\langle\tau\rangle &= t\langle x \mapsto \sigma(x)\langle\tau\rangle\rangle. \end{aligned}$$

They follow by structural induction, with the binder cases using lifted substitutions. Normalization commutes with renaming and introduces no free variables. The contraction procedure and normalized substitution belong to the trusted specification; deleting the zero computation from the inference rules does not delete this implementation cost.

2.2 One evaluation pattern suffices for equality and computation

Use reflexivity and just the following evaluation inference:

$$\frac{\Gamma \vdash I(c, a; u.h) = t \qquad \Gamma \vdash I(Sc, a; u.h) = r}{\Gamma \vdash r = h\langle t\rangle}. \tag{Step}$$

All terms in the display are interpreted by the canonical convention. In particular, at $c = 0$ the first source is simply a. The same c, a, h must occur in both sources. There is one case of this rule, no comparison of a term with an arbitrary context and no search for an occurrence to replace.

Successor computation. Take reflexivity as both premises of Step. Its conclusion is precisely

$$I(Sc, a; u.h) = h\langle I(c, a; u.h)\rangle. \tag{Succ}$$

Zero computation is already the definition of the canonical constructor.

Equality pivot. For an arbitrary canonical term s, abbreviate

$$J_s = I(S0, s; v.v).$$

Given $p : \Gamma \vdash s = t$ and $q : \Gamma \vdash s = r$, Step at $c = 0, a = s, h(v) = v$, with major premise q and minor premise the reflexivity of J_s, proves $J_s = r$. A second instance of the same Step, now with premises p and $J_s = r$, proves $r = t$. Thus

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash s = r}{\Gamma \vdash r = t} \tag{Pivot}$$

is a derived rule using two Steps and one auxiliary reflexivity. This derivation is valid at every s, including literal zero.

Symmetry and transitivity. Pivot applied to $s = s$ and $s = t$ gives $t = s$. Given $s = t$ and $t = r$, first obtain $t = s$ by symmetry and then apply Pivot with common source t to obtain $s = r$. Hence all equivalence-relation laws of equality are available.

Congruence in every fixed term template. Fix a canonical body $h(v)$, allowing passive parameters and nested iterators, and put $K = I(S0, s; v.h(v))$. Step with the reflexivities of s and K proves $K = h\langle s\rangle$. Step with $s = t$ and the reflexivity of K proves $K = h\langle t\rangle$. Pivot gives

$$\Gamma \vdash h\langle s\rangle = h\langle t\rangle.$$

Combining this with $h\langle s\rangle = r$ by Pivot gives the one-template crossed transport rule

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash h\langle s\rangle = r}{\Gamma \vdash r = h\langle t\rangle}. \tag{Tr}$$

Two instances give transport in an equation template. None of these arguments uses equation induction, numerical case distinction, successor injectivity, a logical connective, or a structural term substitution theorem for whole derivations.

Reverse derivation. In the zero-normal native kernel, successor computation gives $I(Sc, a; h) = h\langle I(c, a; h)\rangle$. The two premises of Step, symmetry, transitivity, and term congruence therefore imply $r = h\langle t\rangle$. Consequently Refl and Step are interderivable, with arbitrary side assumptions, with the normalized successor computation and the old one-template equality rule. This is an exact consequence-equivalence theorem, not merely a standard-model soundness observation.

2.3 A normal zero witness for the single introduction

The familiar single-introduction convention takes progress at a literal equation with literal witness 0, and cases otherwise. Its old alternate zero witness $I(0, 0; u.0)$ is no longer available: it is definitionally zero in the canonical grammar. Instead use

$$q := I(S0, 0; u.u).$$

This is a closed canonical term which is not the literal 0. Successor computation proves $q = 0$; equivalently one Step on the reflexivities of 0 and q proves it directly. This syntactic distinction is the only property of the alternate witness needed below.

3 A five-pattern clausal calculus

This subsection gives the full logical interface. It is independent of whether a checker physically stores named variables, de Bruijn indices, or an equivalent scoped encoding.

3.1 Formulas and retained clauses

Formulas have only the constructors

$$A ::= (s = t) \mid D_{x,y}B(x, y).$$

The variables x, y are bound in B. The ordinary interpretation is

$$\left(D_{x,y}B\right)^* := \exists x(B(x, 0)^* \to \forall y\, B(x, y)^*).$$

The designated false formula is $0 = S0$. A retained clause is written

$$[A\,;\, y.B(y)]$$

and has ordinary reading $A^* \to \forall y\, B(y)^*$; its variable y is bound in B and does not occur free in A. A context is a finite list of such clauses. Empty contexts are allowed. For a formula A, its ordinary-assumption encoding is

$$\mathrm{Assm}(A) := [0 = 0\,;\, y.A], \qquad y \notin \mathrm{FV}(A).$$

This is an abbreviation for a clause, not a sixth inference rule.

3.2 Complete rule inventory

The following five patterns, and no further logical or arithmetic rules, are primitive:

$$\text{Use:}\quad \frac{[A; y.B] \in \Gamma \qquad \Gamma \vdash A}{\Gamma \vdash B\langle t\rangle};$$

$$\text{Refl:}\quad \Gamma \vdash t = t;$$

$$\text{Step:}\quad \frac{\Gamma \vdash I(c, a; u.h) = t \qquad \Gamma \vdash I(Sc, a; u.h) = r}{\Gamma \vdash r = h\langle t\rangle};$$

$$\text{DE:}\quad \frac{\Gamma \vdash D_{x,y}B \qquad \Gamma, [B(b,0); y.B(b,y)] \vdash A}{\Gamma \vdash A};$$

$$\text{DI:}\quad \frac{\Gamma, \mathrm{Assm}(B(t,\eta)) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B}.$$

For DE, b is fresh for the outer context, the conclusion, and the passive parameters. For DI, z is fresh for the outer context, the witness t, and all passive parameters; moreover

$$\eta = \begin{cases} z, & B \text{ is a literal equation and } t \text{ is literally } 0, \\ 0, & \text{otherwise.} \end{cases}$$

Every term and formula substitution in these rules is normalized. Fresh variables used in a premise enlarge the scope of the entire outer context as well as the formulas. Omitting that weakening is unsound.

All five patterns have one conclusion. Refl is an axiom schema. Step supplies both computation and equational inference. Use accesses retained clauses and thus includes the ordinary hypothesis rule: from $\mathrm{Assm}(A) \in \Gamma$, apply Use to Refl(0). The only induction condition appears in the syntactic choice of η at DI; no induction for compound matrices is permitted.

3.3 Structural bookkeeping

Context weakening and renaming of free variables are admissible by induction on derivations. Renaming preserves canonicality, literal zero, and the root constructor of a formula, so the DI mode is preserved. Ordinary formula cut is also admissible: if $\Gamma \vdash A$ and $\Gamma, \mathrm{Assm}(A) \vdash C$, then $\Gamma \vdash C$. Replace each Use of that assumption clause by the first derivation, weakened to the current

eigenvariable scope; induction on the second derivation completes the argument. Its antecedent proof of $0 = 0$ can be discarded. Bound eigenvariables are renamed before the replacement when necessary.

These arguments do not require a direct proof of arbitrary term substitution for the syntax-directed DI. Such a substitution may change a nonliteral witness into literal zero. General term substitution becomes admissible after equivalence with the ordinary native kernel has been proved; treating its preservation of the mode as automatic would be an error.

3.4 Conditional closures as formulas

Define the closure abbreviation

$$\widehat{Q}_y B(y) := D_{x,y} B(y), \qquad x \notin \mathrm{FV}(B).$$

Its ordinary reading is equivalent to $B(0) \to \forall y B(y)$ because the numerical domain has the closed inhabitant 0.

Closure elimination. From $\Gamma \vdash \widehat{Q}_y B$ and $\Gamma \vdash B(0)$, apply DE. Its retained clause is $[B(0); y.B(y)]$ and is independent of the fresh witness. Use at any target t, with the weakened base proof, gives $B(t)$. Thus QE is derived.

Closing a retained canonical clause. From a retained clause $[B(0); y.B(y)]$, derive $\widehat{Q}_y B$ by DI with dummy witness q. This is the case mode, since q is not literal zero. Under the local assumption $B(0)$, Use proves $B(Sz)$. The same argument converts any derivation $\Gamma, \mathrm{Assm}(B(0)) \vdash B(Sz)$, with z fresh, into a derivation of $\widehat{Q}_y B$: take the displayed derivation as the premise of DI at dummy witness q.

Equation progress. Given

$$\Gamma, \mathrm{Assm}(E(z)) \vdash E(Sz), \qquad E \text{ a literal equation},$$

apply DI to the dummy-witness matrix $E(y)$ at witness 0. It produces $\widehat{Q}_y E$. Thus the native progress rule is derived with no equality reasoning or additional induction schema.

3.5 Recovering every case introduction

For a compound matrix, or for a witness other than literal zero, DI is already the desired case rule. Only a literal equation $E(x, y)$ with witness 0 requires an argument. Suppose

$$\Gamma, \mathrm{Assm}(E(0,0)) \vdash E(0, Sz).$$

By closing the displayed derivation at the dummy witness q, obtain $\widehat{Q}_y E(0, y)$. Apply DE to retain $[E(0,0); y.E(0,y)]$ and introduce $D_{x,y} E(x, y)$ by DI at witness q, which uses the case mode. In its premise assume $E(q, 0)$. Transport along $q = 0$ gives $E(0, 0)$; Use of the retained clause gives $E(0, Sz)$; transport back along $0 = q$ gives $E(q, Sz)$, as required. The fresh eigenvariable from the outer DE does not occur in the final conclusion. All transports are in literal equation templates and are available from the arithmetic Pivot construction.

Hence the five-pattern calculus derives both original introduction rules, including the otherwise missing atomic case at zero. This proof also explains why normalizing the old nonliteral zero witness without replacing it is insufficient.

4 Exact equivalence and the inherited iPRA bridge

Let N_0 denote the original native calculus on raw scalar terms, and let N_z be its zero-normal presentation, with ordinary Hyp, successor computation, one-template transport, QE, DE, case DI, and atomic QI. The pinned baseline proves

$$N_0 : \Gamma \vdash A \quad \Longleftrightarrow \quad N_z : N\Gamma \vdash NA.$$

Its formal source is `NativeZeroNormal.lean`; the normal-form coherence and interpretation facts are in `ZeroNormalization.lean`. The semantic fact that a raw term is provably equal to its normal form under an iterator body uses body congruence, which in that inherited development uses equation induction. The new equality bootstrap above does not claim to avoid that inherited requirement in the proof of the normalization theorem.

Define T on the formulas of N_z by preserving equality and D, and replacing every $Q_y B$ by $\widehat{Q}_y(TB)$. Translate ordinary hypotheses A to $\mathrm{Assm}(TA)$. The translation T commutes with normalized numerical substitution up to renaming bound variables: prove this by structural induction on formulas, choosing each dummy witness binder fresh for the substitution images. The preceding derivations translate every rule of N_z into the five-pattern calculus C: Hyp becomes Use with Refl(0); successor computation and transport are derived from Step; QE uses DE and Use; equation progress is DI at a dummy zero witness; every case introduction is recovered as above. In the translation of DE, the retained canonical clause derives the translated hypothetical closure, so admissible formula cut supplies the translated branch derivation.

For the reverse direction, interpret retained clauses as $A^* \to \forall y B(y)^*$ in the ordinary scalar theory T_{eq}. Use is implication elimination followed by universal elimination. Refl is equality reflexivity. Step follows from successor computation and equality. DE is ordinary existential elimination. In case-mode DI, coverage $n = 0 \vee \exists z\, n = Sz$ combines the base assumption and the successor premise to prove the conditional closure. In progress-mode DI, contextual induction for the literal equation supplies that closure. Existential introduction then supplies the stated witness in each mode. Thus the entire calculus is sound in T_{eq} with precisely the same domain and actual equality.

The pinned ordinary/native equivalence and the rule translations yield, for every finite list Γ of native formulas and every native A,

$$N_0 : \Gamma \vdash A \quad \Longleftrightarrow \quad C : \mathrm{Assm}(T(N\Gamma)) \vdash T(NA). \qquad \text{(Faithfulness)}$$

Here the right-hand context means pointwise application to its entries. The reverse implication uses that the ordinary reading of $\mathrm{Assm}(B)$ is equivalent to B, and that the ordinary reading of the dummy-existential closure is equivalent to Q. Those equivalences are intuitionistically valid over an inhabited domain.

Bi-interpretability, rather than only equiconsistency. The inherited bridge interprets raw scalar terms as their fixed primitive recursive definitions in iPRA, and compiles each fixed primitive recursive definition back to a scalar term. Its round trips are provably the identity on numerical values. Compose those maps with N and T. The normalizing map fixes variables and zero,

preserves numerical values provably, and leaves the whole numerical domain unchanged. The closure replacement has no numerical effect. Consequently both composites retain the identity graph $x = y$; equality remains actual equality, and no quotient or proper subdomain is introduced.

On formulas, inclusion followed by $T \circ N$ is the identity on the canonical Eq/D language. The other round trip is provably equivalent to the input formula by zero-normalization equivalence and $Q_y B \leftrightarrow D_{x,y} B$ for a dummy x. Transporting the inherited ordinary/iPRA formula round-trip theorems through Faithfulness gives both full first-order round trips for C. In particular, the designated contradiction $0 = S0$ is preserved, so the resulting theories are equiconsistent as well as bi-interpretable.

Boundary of the claim. The new mathematical deletion is the replacement of the multi-case arithmetic equality/computation mechanism by Refl and one uniform Step. Zero-normalization and clausal logical contexts are inherited design ideas and their implementation costs must be counted. A numerical size comparison is a separate claim about complete programs, not a theorem that no smaller presentation exists. Any proof-assistant status must be reported according to the files actually compiled; the written five-pattern equivalence is not automatically a machine-checked theorem.

5 Complete verifier and certificate contract

The primary module `code/checker.py` is the entire trusted acceptance program. It imports only Python's standard `json` and `sys` modules. It reads one JSON object, validates the closed goal, reconstructs the proof conclusion, and accepts exactly when they agree. Acceptance is exit status zero and the line `ACCEPT`. Rejection has nonzero exit status. Python's resource limits can reject sufficiently large valid inputs; no resource-bound completeness claim is made.

5.1 Concrete syntax

The public object has exactly two fields, `goal` and `proof`. All term and formula trees are explicitly checked:

Mathematical object	JSON form	Binding
Zero	`0`	None
Successor	`['s', a]`	None
Iterator	`['i', n, a, h]`	One variable in `h`
Equation	`['=', a, b]`	None
Paired formula	`['d', B]`	Two variables in `B`

The display uses single quotes for readability; JSON files use double quotes. A positive integer denotes a bound-variable index, with index 1 the innermost variable. In a D matrix, index 1 denotes y and index 2 denotes x. Negative integers are rigid free-variable levels used only inside local proof scopes. At free scope k and binder depth d, an integer term must satisfy $-k \leq a \leq d$. The number zero always denotes zero, not a variable. Python booleans are explicitly excluded by exact integer-type tests.

Canonical term validation forbids an iterator with literal zero counter, recursively everywhere. The shared opening operation rebuilds every visited iterator using the canonical constructor, so exposing

a zero counter during substitution immediately contracts it. The test is on the rebuilt counter, not on its original raw syntax.

The proof constructors are as follows:

Rule	Certificate form
Use	`['u', clause-index, witness, antecedent-proof]`
Refl	`['r', term]`
Step	`['x', major-proof, minor-proof]`
DE	`['e', goal, paired-proof, branch-proof]`
DI	`['i', paired-goal, witness, branch-proof]`

The context is an internal list of antecedent/body pairs; it is not an unchecked public environment. A DE branch opens its witness at the next fresh negative level. A DI branch uses the next fresh level as its induction/case parameter. The conclusion and chosen witness are validated in the old scope before either branch opens, which prevents an eigenvariable from escaping. Ordinary local assumptions are stored as clauses with antecedent $0 = 0$.

5.2 Why the opening operation needs no shift

The use of negative levels is an implementation convention, not an extra axiom. Every substitution image is locally closed: all its positive indices are bound within that image itself, and all its external variables have negative labels. Inserting such an image under an additional binder therefore cannot capture an external variable. The opening traversal increases its depth under the iterator body and by two under D, replaces the designated bound index, and decrements larger indices. The substituted image is not recursively opened a second time.

This argument would fail for arbitrary open positive-index images. The validator prevents that input. The proof-reconstruction invariant is that every returned formula is canonical and well-scoped in the current scope, and every retained clause has a well-scoped antecedent and a body with exactly one additional permitted bound variable. Each constructor preserves that invariant. In particular, Step's premises are explicitly matched as equations with the required successor-iterator source.

5.3 Measurements

All source measurements count the entire acceptance module. For comparison with the supplied repository, module, class, and function docstrings are removed from the AST, then the source is normalized by CPython 3.12.14 `ast.unparse`, followed by one LF. Significant tokens exclude comments, whitespace, indentation, newline, encoding, and end markers. AST nodes are counted after that same docstring removal. Gzip uses maximum level 9 and timestamp zero; it is informational, not an optimization objective. For the delivered primary Python file itself, including its documentation and original formatting, the corresponding figures are 3507 source bytes and 1274 gzip-9 bytes. Both raw and normalized compressed files are included.

Complete checker	Bytes	Tokens	AST	Gzip 9
Matched zero-normal Comp/Tr	3048	860	949	1018
Matched raw three-case Eval	3094	853	969	1026
Incoming canonical clausal	3305	946	1072	1073
New uniform Step	2950	829	902	1005

The matched canonical control shares the terms, normalization, opening operation, formula grammar, public fields, validation, clause protocol, and logical rules with the new checker. Its arithmetic rules are successor computation and one-template crossed transport. Consequently that comparison isolates the arithmetic replacement. The raw Eval control retains raw zero-counter iterators and the three arithmetic Eval alternatives. Its comparison also includes the cost of adding canonical syntax.

The secondary modules in `code/native/` keep the more general explicit-sequent Eq/Q/D interface with a variable count and public hypotheses. There the matched normalized control measures 3,118 bytes, 1,010 tokens, and 1,080 AST nodes; Refl/Step measures 3,029 bytes, 981 tokens, and 1,051 AST nodes. The pinned raw Eval checker measures 3,039 bytes, 959 tokens, and 1,026 AST nodes. Thus the change has a clear matched win, but its comparison with raw Eval in that second interface is mixed. These modules are supplementary evidence, not additional hidden dependencies of the primary verifier.

5.4 Verification evidence and its limits

- Primary clausal suite: 376 standalone-process checks in ordinary and optimized Python; no failures.
- Explicit-sequent suite: 38 positive certificates, 12 negative controls, and 38 complete proof-translation round trips; 100 command-line checks.
- Independent opening audit: 2104 semantic comparisons, 13 targeted malformed/capture controls, and one positive smoke check.

The explicit-sequent suite supplies actual finite proof translations in both directions between normalized computation/transport and Refl/Step. The translators are untrusted producers: each output is checked by the target's complete verifier. They cover every primitive constructor of those calculi, and preserve the public variable count, context, and goal.

The tests specifically address the difficult implementation boundaries: an induction hypothesis that is actually used, a closure actually used after D-elimination, nested term binders, normalization exposed by substitution, arbitrary equation assumptions, malformed indices, booleans, forged predecessor sources, and escaping eigenvariables. An independently written numerical evaluator also checks the opening operation on finite instances. Numerical tests are supplementary and are not used to infer the nonstandard-model interpretation theorem.

The script `reproduce.py` reruns the validation suites and recomputes all source metrics and deterministic gzip files. Its optional `-pdf` switch rebuilds the standalone `report.tex` and compiles the PDF. The inherited reference files are byte-authenticated against Git blob identifiers at the frozen commit; `evidence/provenance.json` records that check.

6 Full first-order interpretation and research boundary

The interpretation into ordinary logic uses the entire numerical domain and interprets each native equality as actual equality. The D binder and clauses have precisely the ordinary readings specified above. Genuine ordinary falsity is interpreted as $0 = S0$. The ordinary scalar realization explicitly includes the separation axiom $(0 = S0) \to \bot$; positive numerical equations alone do not supply this ordinary falsity bridge.

For transparency, the complete inherited logical macro construction is reproduced in Appendix B. It derives implication, conjunction, disjunction, both quantifiers, and explosion using only selectors, finite iteration, and the native rules. It also proves the paired-formula round trip

$$D_{x,y}B \quad \dashv\vdash \quad \exists_* x\big(B(x,0) \Rightarrow_* \forall_* y\, B(x,y)\big).$$

This is why the result concerns full intuitionistic first-order arithmetic rather than just an equational or negative fragment. No excluded middle for quantified formulas is introduced.

The arithmetic bridge is also internal. Its order matters: first construct predecessor by scalar iteration and equation induction; then prove faithful equality reflection; then obtain quantifier-free induction; only then construct pairing and compile general primitive recursion. The scalar update has one bound state variable and no hidden recursion-counter argument.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

the inherited pairing π and projections L, H give

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$, and the projection equations prove both defining equations for f^*. Each fixed finite primitive-recursive description is compiled externally. There is no internally total universal evaluator over arbitrary descriptions.

In the other direction, each fixed scalar iterator is an ordinary primitive recursion ignoring its counter in the update. Equation induction proves uniqueness for these fixed recursions, so an external induction over descriptions proves both internal numerical round trips. Normalization preserves numerical values, and the clause translation does not change numerical terms. This establishes the identity graph $x = y$ on the whole domain in both composites, including nonstandard elements.

6.1 What remains open

The five-rule inventory retains a syntax-directed case/progress choice inside DI. It has not removed the substantive distinction between numerical coverage and equation induction. Further merging the logical rules would need to preserve that distinction and its eigenvariable conditions.

The result also exchanges a larger equality interface for a modest increase in some proof sizes: derived equality composition expands to two Step nodes and an auxiliary reflexivity. Checker source size, certificate length, and checking time are separate objectives. The implementation measurements do not establish an optimum for any of them.

The next useful formal-verification target is a deeply embedded Step calculus and its equivalence with `ConciseIPRA.NativeZeroNormal`. The present package provides the necessary derivations and tested finite translations, but does not label an uncompiled Lean sketch as a verified result.

A Inherited proof material and attribution

The next five appendices reproduce the exact mathematical source fragments from `Research/certified_zero/reference/` at the frozen commit. The comparison script verifies their Git blob hashes. They are included under the repository's MIT-0 license, retained as `reference/LICENSE-ProveIt.txt`. Their descriptions of earlier contributions are relative to their original reports. They do not assert priority for this package.

These fragments use the raw scalar notation and their own earlier native rule presentation. Sections 2 through 4 prove the equivalence that allows their arithmetic bridge to be used here. In particular, the fragments' full formula-transport rule is a recovered rule, not an additional primitive of the five-rule calculus.

B Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

B.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)} \quad \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0, 1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t, 0), [B(t, z)]_{\mathrm{at}} \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)}. \tag{Old DI}$$

The bracket supplies $B(t, z)$ exactly at a literal equation.

B.2 Exact equivalence and the unused-witness construction

Lemma B.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem B.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x, y)$ and its transformed premise is

$$\pi : \quad \Gamma, E(t, 0), E(t, z) \vdash E(t, Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t, Sz)$ under $\Gamma, E(t, 0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t, y)$ and all retained data. Under $\Gamma, E(t, 0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y}E(t, y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t, 0); y.E(t, y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t, 0)$ supplies its antecedent. Use at Sz gives $E(t, Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x, y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x, y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t, y)$ and binds only the unused coordinate u.

Proposition B.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

B.3 Induction and numerical cases separate cleanly

Lemma B.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma B.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D *are derivable without using the progress choice.*

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

B.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A,B) \dashv\vdash A$ and $M_{Sv}(A,B) \dashv\vdash B$.

Put $\delta(v) = C(v,1,0)$ and $p(z) = I(z,0;u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z} M_z(A,B), \\ A \wedge_* B &:= \mathsf{D}_{u,z} G_z M_{p(z)}(A,B), \\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y} G_y A(x), \\ A \vee_* B &:= \exists_* x\, M_x(A,B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z} G_z A(z), \\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A,B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A,B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A,B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0,s,t) = C(1,s,t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

B.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x,0) \to \forall y B^\circ(x,y)\big),$$

$$\langle A; y.B\rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x\big(B(x,0) \Rightarrow_* \forall_* y\, B(x,y)\big). \qquad \text{(Paired composite)}$$

Forwards, DE opens its clause at a fresh a; under $B(a,0)$ that clause supplies $B(a,y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem B.2 preserves every native sequent.

B.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

C Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section B recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

C.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.

4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u),\vec{p}))\big), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem C.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

D Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [2] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

D.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

D.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n)+1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n + 2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

D.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

D.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \tag{12}$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \tag{13}$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \tag{14}$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \tag{15}$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

D.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

E From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [3]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix D, the selector is $C(n,a,b) = I(n,a;u.b)$, addition is $x+y = I(y,x;u.Su)$, and truncated subtraction is $d(x,y) = I(y,x;u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

E.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$

$$C(0,a,b) = a, \qquad C(Sx,a,b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x\,(x = 0 \vee x = S(Px)), \tag{18}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n,\vec{p})$:

$$\forall \vec{p}\,\left[(E(0,\vec{p}) \wedge \forall n(E(n,\vec{p}) \longrightarrow E(Sn,\vec{p}))) \longrightarrow \forall n\, E(n,\vec{p})\right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0,\vec{p}), \qquad \Gamma, E(z,\vec{p}) \vdash E(Sz,\vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n,\vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

E.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u, 0, 1), \qquad K(u) = C(u, 1, Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u, v, v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

E.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$
$$(Sx) + y = S(x + y), \tag{32}$$
$$d(Sx, Sy) = d(x, y), \tag{33}$$
$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

E.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C\big(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)\big) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}\big(K(d(Sa, b))\big). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$

$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{40}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (25), which is equivalent to $d(Sa, b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (39) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, \big(a = b \vee (a = b \to \bot)\big) \tag{41}$$

is a theorem, not an extra hypothesis.

E.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s,t), \qquad b_\bot = 1, \tag{42}$$
$$b_{A\wedge D} = C(b_A, b_D, 1), \tag{43}$$
$$b_{A\vee D} = C(b_A, 0, b_D), \tag{44}$$
$$b_{A\to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$\big(b_A = 0 \vee b_A = 1\big) \quad \text{and} \quad \big(b_A = 0 \longleftrightarrow A\big). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

E.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

F Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [4]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

F.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a,b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n)+1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a,b) = (a+b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a,b)) = a + b$, and therefore

$$L(\pi(a,b)) = a, \qquad H(\pi(a,b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

F.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u,\vec{p}) = \pi(SL(u), h^*(L(u), H(u),\vec{p})),$$

$$V(n,\vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u,\vec{p})), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n,\vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

F.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section B give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

G Complete primary verifier

The following listing is the complete primary acceptance module. The normalization and source-size procedures do not remove executable code from it.

```
"""Five-rule clausal iPRA with zero-normal iterators and uniform successor Step.

Numerical syntax: 0; integer bound indices (>0) or free levels (<0);
['s', term]; ['i', counter, seed, body]. Only body binds one variable.
Formula syntax: ['=', left, right]; ['d', matrix]. D binds x then y,
so the matrix uses index 1 for y and 2 for x. Zero-counter iterators
are excluded, and opening contracts them when substitution exposes one.
"""
import json
import sys


def require(condition):
    if not condition:
        raise ValueError('Invalid certificate')


def iterator(n, a, h):
    return a if n == 0 else ['i', n, a, h]


def term(a, scope, depth=0):
    if type(a) is int:
        return -scope <= a <= depth
    if type(a) is not list:
        return False
    if len(a) == 2 and a[0] == 's':
        return term(a[1], scope, depth)
    return (len(a) == 4 and a[0] == 'i' and a[1] != 0
            and term(a[1], scope, depth) and term(a[2], scope, depth)
            and term(a[3], scope, depth + 1))


def formula(a, scope, depth=0):
    return (type(a) is list
            and (len(a) == 3 and a[0] == '='
                 and all(term(t, scope, depth) for t in a[1:])
                 or len(a) == 2 and a[0] == 'd'
                 and formula(a[1], scope, depth + 2)))


def opening(a, value, depth=0):
    if type(a) is int:
        return value if a == depth + 1 else a - (a > depth + 1)
    tag = a[0]
    b = [opening(t, value, depth + (2 if tag == 'd' else int(tag == 'i' and i == 2)))
         for i, t in enumerate(a[1:])]
    return iterator(*b) if tag == 'i' else [tag] + b


def derive(proof, context, scope):
    match proof:
        case ['u', index, witness, premise]:
            require(type(index) is int and 0 <= index < len(context)
                    and term(witness, scope))
            antecedent, body = context[index]
            require(derive(premise, context, scope) == antecedent)
            return opening(body, witness)
        case ['r', a]:
            require(term(a, scope))
            return ['=', a, a]
        case ['x', major, minor]:
```

```
            match derive(major, context, scope), derive(minor, context, scope):
                case ['=', s, t], ['=', ['i', ['s', n], a, h], r]:
                    require(s == iterator(n, a, h))
                    return ['=', r, opening(h, t)]
        case [('e' | 'i') as tag, goal, data, branch]:
            require(formula(goal, scope))
            fresh = -scope - 1
            if tag == 'e':
                paired = derive(data, context, scope)
                require(paired[0] == 'd')
                body = opening(paired[1], fresh, 1)
                clause = (opening(body, 0), body)
                target = goal
            else:
                require(goal[0] == 'd' and term(data, scope))
                body = opening(goal[1], data, 1)
                eta = fresh if goal[1][0] == '=' and data == 0 else 0
                clause = (['=', 0, 0], opening(body, eta))
                target = opening(body, ['s', fresh])
            require(derive(branch, context + [clause], scope + 1) == target)
            return goal
    raise ValueError('Invalid proof')


def verify(data):
    require(type(data) is dict and set(data) == {'goal', 'proof'})
    require(formula(data['goal'], 0))
    require(derive(data['proof'], [], 0) == data['goal'])
    return True


if __name__ == '__main__':
    try:
        verify(json.load(sys.stdin))
    except Exception:
        sys.exit(1)
    print('ACCEPT')
```

References

[1] Vladimir Reshetnikov, *ProveIt: Concise presentations of intuitionistic primitive recursive arithmetic*, commit `6ca328039cafa1537d81ee6b67ee2dc54ffa5a4e`. ConciseIPRA at the frozen comparison commit.

[2] ProveIt research package, *Internal scalar predecessor bootstrap*, `Research/certified_zero/reference/scalar_predecessor.tex`, at the commit in [1]. Reproduced with attribution.

[3] ProveIt research package, *From literal-equation induction to full intuitionistic PRA*, `Research/certified_zero/reference/equality_reflection.tex`, at the commit in [1]. Reproduced with attribution.

[4] ProveIt research package, *Pairing, primitive recursion, and internal numerical composites*, `Research/certified_zero/reference/primitive_recursion.tex`, at the commit in [1]. Reproduced with attribution.

[5] ProveIt research package, *Complementary introductions with one local assumption*, `Research/certified_zero/reference/inherited_choice_basis.tex`, and *Arithmetic strength and both ordinary interpretation composites*, `inherited_ordinary_bridge.tex`, at the commit in [1]. Reproduced with attribution.

[6] ProveIt, *ZeroNormalization* and *NativeZeroNormal*, Lean sources under `Lean/ConciseIPRA/`, at the commit in [1].

[7] ProveIt, *Computational equality: six rules*, `Research/computational_equality/` and `Lean/ConciseIPRA/NativeEval.lean`, at the commit in [1].

[8] ProveIt incoming archives, `docs/incoming/ipra_clausal_canonical.zip` and `docs/incoming/ipra_scalar_uniqueness.zip`, at the commit in [1]. The former supplies the incoming five-rule comparison; the latter supplies the 866-byte profile mentioned only for scope of comparison.